PDF DOCUMENT ASSISTANT USING RETRIEVAL-AUGMENTED GENERATION (RAG)

Project Report | Prepared by K G SHRUJANA

1. Abstract

The PDF Document Assistant is an AI-powered application that helps users retrieve information from PDF documents by asking questions in natural language. Instead of manually searching through every page, a user uploads a document, processes it, and asks a question. The system extracts the text, divides it into smaller chunks, converts the chunks into numerical embeddings, and indexes them for efficient similarity search. When a question is submitted, the application retrieves relevant chunks and sends them as context to a language model through the Groq API. The generated answer is displayed through a Streamlit web interface. The project combines document processing, semantic retrieval, and language generation to make document-based information access faster and more convenient.

2. Introduction

PDF files are widely used for academic notes, reports, manuals, and reference materials. However, long documents can be time-consuming to read when a user needs only a particular fact or explanation. A keyword search may also miss relevant passages when the wording of a question differs from the wording in the document. This project addresses that problem by providing a question-answering interface over an uploaded PDF. It uses Retrieval-Augmented Generation (RAG): first retrieve relevant content from the document, then use that content to help a language model produce a response.

3. Problem Statement

Users often spend considerable time locating specific information in lengthy PDF documents. The objective is to develop a simple application that allows users to upload a PDF and ask questions about its content, reducing manual searching and making document information easier to access.

4. Objectives

- Allow users to upload a PDF through a simple web interface.
- Extract readable text and split it into manageable, overlapping chunks.
- Represent chunks as embeddings and use FAISS to retrieve relevant content.
- Use retrieved content as context for AI-generated answers through the Groq API.
- Provide an interface using Streamlit and support repeated questions about a processed document.

SYSTEM DESIGN AND IMPLEMENTATION

PDF Document Assistant | K G SHRUJANA

5. System Architecture and Workflow

The application follows a sequence of processing stages. First, the user uploads a PDF. The document loader extracts text from its pages. The text is divided into smaller chunks so individual passages can be searched efficiently. Sentence Transformers converts each chunk into an embedding, a numerical representation that can be compared with other embeddings. These vectors are stored in a FAISS index. When a user enters a question, the question is embedded and compared against the indexed chunks. The most relevant chunks are retrieved and included in a prompt sent to a language model through Groq. The generated response is returned to the Streamlit interface for display.

Main Workflow

- **PDF Upload:** The user selects a readable PDF file.
- **Text Extraction:** The document loader reads available text from the PDF.
- **Chunking:** The extracted text is split into smaller overlapping passages.
- **Embedding and Indexing:** Sentence Transformers creates embeddings and FAISS stores searchable vectors.
- **Retrieval:** The system searches for chunks most relevant to the user's question.
- **Answer Generation:** Retrieved passages are provided as context to the Groq-hosted language model.
- **Display:** Streamlit shows the generated answer to the user.

6. Technologies Used

- **Python:** Core programming language connecting the application's components.
- **Streamlit:** Builds the interface for PDF uploads and question submission.
- **PyPDF:** Extracts selectable text from PDF documents.
- **Sentence Transformers:** Converts text into numerical embeddings for similarity-based retrieval.
- **FAISS:** Enables efficient similarity search over stored vectors.
- **Groq API:** Provides access to a language model that generates answers using retrieved context.
- **RAG:** Combines document retrieval with AI-based answer generation.

7. Main Project Components

- **app.py:** Main Streamlit application connecting user actions with document processing and question answering.
- **document_loader.py:** Handles PDF text extraction.
- **vector_store.py:** Creates and manages embeddings and the FAISS search index.
- **rag_pipeline.py:** Coordinates retrieval and answer generation.
- **prompt.py:** Defines prompt instructions used to guide the language model.
- **requirements.txt:** Lists the Python packages needed to run the application.
- **tests:** Contains tests for chunking, PDF loading, and retrieval behaviour.

TESTING, RESULTS AND CONCLUSION

PDF Document Assistant | K G SHRUJANA

8. Implementation and Execution

The project is implemented as a Python application with a Streamlit interface. After installing dependencies listed in requirements.txt and configuring the Groq API key in a local .env file, the application can be started with `streamlit run app.py`. The user opens the local web page, uploads a PDF, and selects the document-processing action. Once processing finishes, the user enters a question and submits it. The system retrieves relevant text chunks and uses them to construct the context for answer generation. The response is displayed in the application. The API key should be kept private and must not be included in screenshots, recordings, or the submitted report.

9. Testing and Expected Results

The project includes separate tests for PDF text loading, chunking, and search/retrieval. Useful checks include uploading a readable PDF, verifying that text is extracted, processing a document containing a known answer, asking a question about that answer, and comparing the generated response with the original PDF. A second question can verify that the processed document supports repeated queries. Empty PDFs, scanned documents without selectable text, invalid files, missing API configuration, and questions unrelated to the document should also be considered. Actual results depend on the input PDF, retrieval quality, and language model response.

10. Advantages

- Reduces the time required to search lengthy documents manually.
- Allows users to ask questions in natural language.
- Uses retrieved document passages to make answers more relevant to the uploaded PDF.
- Combines a simple interface with modular Python components.
- Can be used with academic notes, reports, manuals, and other text-based PDFs.

11. Limitations and Future Enhancements

The application depends on readable text in the PDF; scanned or image-only documents may require Optical Character Recognition (OCR). Retrieval may miss useful passages, and a language model can still produce an incorrect or incomplete answer. The Groq API requires valid configuration and network access. Future improvements could include showing page references for answers, adding OCR, improving handling of questions that are not answered in the document, evaluating retrieval quality with a test set, and supporting multiple document collections.

12. Conclusion

The PDF Document Assistant demonstrates how Retrieval-Augmented Generation can make document-based question answering more convenient. It combines PDF text extraction, chunking, embeddings, FAISS similarity search, and language-model generation behind a Streamlit interface. The project provides a practical way to retrieve information from long PDFs and illustrates the integration of Python and modern AI components. Although answer quality depends on the document and retrieval process, the application provides a useful foundation for more accurate and user-friendly document assistants.

— End of Report —